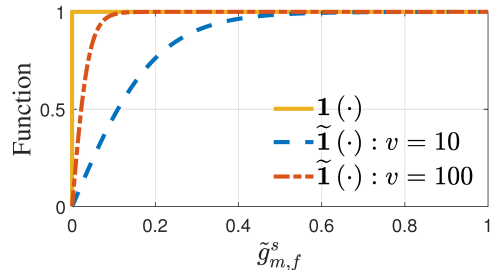

Fig. 2. Impact of different values of v on smoothing function $\tilde{\mathbf{1}}(\cdot)$.

to address constraints in (14b) and (14c). The primal-dual problem is expressed as

$$\max_{\lambda^s_m} \min_{p^L_{m,f}, p^S_{m,f}} \sum_{f=1}^{F} \mathbf{1}(g_f) + \sum_{s\in\{L,S\}} \sum_{m=1}^{M} \lambda^s_m c^s_m \tag{15a}$$

$$\text{s.t.} \quad \lambda^s_m \geq 0, m = 1, \ldots, M, s \in \{L, S\}, \tag{15b}$$

$$(14d), (14e),$$

where λ^L_m and λ^S_m are the multipliers associated with constraints in (14b) and (14c), respectively, and $c^L_m \triangleq R^L_m - LB\sum_{f=1}^{F}\log(1+\gamma_{m,f}g^L_{m,f})$ and $c^S_m \triangleq R^S_m - LB\sum_{f=1}^{F}\log(1+\gamma_{m,f}g^S_{m,f}) + \sqrt{N^S_m LB/\tau}Q^{-1}(\epsilon_m)$ are defined based on the QoS constraints in (14b) and (14c).

B. Adaptive Smoothing Functions

The piecewise maximal function $g(\cdot)$ and the indicator function $\mathbf{1}(\cdot)$ involved in problem (15) are non-differentiable. These two functions pose obstacles for the gradient-based training of DNNs since their gradients are either zero or infinity, causing DNNs converging to poor solutions or not converging at all. To deal with the non-differentiable functions, we develop two smoothing functions, denoted as $\tilde{g}(\cdot)$ and $\tilde{\mathbf{1}}(\cdot)$, to approximate the original functions $g(\cdot)$ and $\mathbf{1}(\cdot)$, respectively. $\tilde{g}(\cdot)$ is a variation of SoftMax function and $\tilde{\mathbf{1}}(\cdot)$ is a variation of Tanh function, which are defined as

$$\tilde{g}^s_{m,f} \triangleq \tilde{g}\big(p^s_{m,f}, \mathcal{P}^s_{m,f}, u\big) = \frac{p^s_{m,f}}{\sum_{i\in\{L,S\}}\sum_{j=1}^{M} e^{u(p^i_{j,f}-p^s_{m,f})}}, \tag{16a}$$

$$\tilde{\mathbf{1}}\left(\tilde{g}^s_{m,f}, v\right) = \frac{2}{1+e^{-v\tilde{g}^s_{m,f}}} - 1, \tag{16b}$$

where $u, v \geq 0$ are the smoothing parameters. The values of u and v affect the approximation errors. It is not difficult to find that when u and v are infinite, the approximation errors become zeros. Meanwhile, u and v also affect the smoothness, i.e., gradient, of the approximation functions, as shown in Fig. 2. Increasing the smoothing parameters can improve the approximation accuracy but simultaneously reduce the smoothness, and vice versa.

Existing methods either adopt fixed smoothing parameters or simply adjust them as the number of iterations increases, such as the annealing technique. These methods assume a one-size-fits-all parameter setting that is intended to be effective when the variable in a smoothing function takes different values, which however is difficult if not impossible. For example, considering the smoothed indicator function $\tilde{\mathbf{1}}(\tilde{g}^s_{m,f}, v)$ in (16b), as shown in Fig. 2, using a larger smoothing parameter results in the gradient $\nabla_{\tilde{g}^s_{m,f}}\tilde{\mathbf{1}}(\tilde{g}^s_{m,f}, v)$ approaching zero for large values of $\tilde{g}^s_{m,f}$, thereby resulting in gradient vanishing issues. Conversely, a smaller smoothing parameter introduces large approximation errors for small values of $\tilde{g}^s_{m,f}$. During the training process, the input of the smoothing functions may take different values among iterations, making it difficult to find an appropriate smoothing parameter.

We propose to adaptively adjust the smoothing parameters v and u in $\tilde{\mathbf{1}}(\cdot)$ and $\tilde{g}(\cdot)$. Since it is difficult to find a universal parameter suitable for all variable values, we let the values of v and u change with the variable values. Inspired by the annealing technique, in the early iterations of training, smaller smoothing parameters are preferred to avoid gradient vanishing or exploding, facilitating faster convergence, while as the number of iterations increases, reducing the approximation errors becomes more crucial. Thus, we let the values of v and u also adjust with the number of iterations.

Next we elaborate the design of the adaptive smoothing parameters for the two functions.

1) Smoothing indicator function: With the smoothing functions in (16b), the indicator functions used in problem (15), i.e., $\mathbf{1}(g_f)$ and $\mathbf{1}(g^S_{m,f})$, can be approximated by $\tilde{\mathbf{1}}(\tilde{g}_f, v)$ and $\tilde{\mathbf{1}}(\tilde{g}^S_{m,f}, v)$, respectively, where $\tilde{g}_f = \sum_{m=1}^{M}\tilde{g}^L_{m,f} + \tilde{g}^S_{m,f}$. For ease of discussion, we take $\tilde{\mathbf{1}}(\tilde{g}, v)$ as an example to elaborate the parameter adaptation, where subscript f is omitted.

In order to prevent DNNs from converging to a poor solution or failing to converge due to the gradients that are either too large or too small, we adjust the smoothing parameter v for each value of $\tilde{g}$ to make the absolute value of gradient $|\nabla_{\tilde{g}}\tilde{\mathbf{1}}(\tilde{g}, v)|$ approach a predetermined value (called gradient requirement in the sequel). Moreover, this gradient requirement is not fixed, which evolves during training. At the start, a larger gradient can accelerate network convergence, whereas a reduced gradient can decrease approximation errors as training progresses. Therefore, we let the gradient requirement decrease as the number of iterations increases.

Now given an iteration during the training process, for which the gradient requirement has a value of V, let us examine how to determine the corresponding value of v. Taking into account both the approximation error and gradient requirement, the problem of finding the smoothing parameter v can be formulated as minimizing the approximation error under the gradient requirement, which is

$$\min_{v\geq 0} \left|\tilde{\mathbf{1}}(\tilde{g}, v) - \mathbf{1}(\tilde{g})\right| \tag{17a}$$

$$\text{s.t.} |\nabla_{\tilde{g}}\tilde{\mathbf{1}}(\tilde{g}, v)| = V. \tag{17b}$$

As proven in Appendix B, problem (17) is feasible if $\frac{2\zeta e^{-\zeta}}{\tilde{g}(1+e^{-\zeta})^2} > V$ holds, in which case the optimal solution of v can be obtained by using bisection search on the equation $|\nabla_{\tilde{g}}\tilde{\mathbf{1}}(\tilde{g}, v)| = V$, where $\zeta \approx 1.54$ is a constant. If problem (17) is infeasible, i.e., the required gradient V cannot be achieved, then we find the smoothing parameter v that can maximize $|\nabla_{\tilde{g}}\tilde{\mathbf{1}}(\tilde{g}, v)|$, whose solution is $\frac{\zeta}{\tilde{g}}$ as derived in Appendix B.

To summarize, the smoothing parameter v is computed for every iteration based on the current value of $\tilde{g}$. The obtained v ensures that the absolute value of gradient $|\nabla_{\tilde{g}}\tilde{\mathbf{1}}(\tilde{g}, v)|$ is close to the gradient requirement V, which can avoid the vanishing or exploding of gradient. The value of V decreases as the number of iterations increases, resulting in gradually reduced approximation errors. Similarly, the smoothing parameter for function $\tilde{\mathbf{1}}(\tilde{g}_m^S, v)$ can also be adaptively adjusted during the training process, considering both the number of iterations and the specific value of $\tilde{g}_m^S$.

2) Smoothing piecewise maximal function: The smoothing piecewise maximal function $\tilde{g}_{m,f}^s$ in (16a) is a multivariate function of the power allocation vector $\boldsymbol{p}_f = [p_{1,f}^L, \cdots, p_{M,f}^L, p_{1,f}^S, \cdots, p_{M,f}^S] \in \mathbb{R}^{1\times 2M}$. We assign different smoothing parameters for each element of $\boldsymbol{p}_f$, denoted as $\boldsymbol{u} = [u_1^L, \cdots, u_M^L, u_1^S, \cdots, u_M^S] \in \mathbb{R}^{1\times 2M}$. Then, the expression of $\tilde{g}_{m,f}^s$ in (16a) can be redefined with $\boldsymbol{u}$ as

$$\tilde{g}\big(p_{m,f}^s, \mathcal{P}_{m,f}^s, \boldsymbol{u}\big) = \frac{p_{m,f}^s}{\sum_{i\in\{L,S\}}\sum_{j=1}^M e^{u_j^i(p_{j,f}^i - p_{m,f}^s)}}. \quad (18)$$

For notational simplicity, the subscript f is omitted in the sequel of this subsection.

The design of the smoothing parameters $\boldsymbol{u}$ should take into account both the approximation error and gradient requirement. With (13) and (18), the absolute value of approximation error $|\tilde{g}_m^s - g_m^s|$ can be obtained as

$$\begin{cases} p_m^s - \frac{p_m^s}{\sum_{i\in\{L,S\}}\sum_{j=1}^M e^{u_j^i(p_j^i - p_m^s)}}, & p_m^s > \max\{\mathcal{P}_m^s\}, \\ \frac{p_m^s}{\sum_{i\in\{L,S\}}\sum_{j=1}^M e^{u_j^i(p_j^i - p_m^s)}}, & p_m^s < \max\{\mathcal{P}_m^s\}. \end{cases} \quad (19)$$

The gradient $\nabla_{\boldsymbol{p}}\tilde{g}_m^s$ is a vector, and we employ the ℓ_1 norm of $\nabla_{\boldsymbol{p}}\tilde{g}_m^s$ to measure the gradient magnitude, i.e., $\|\nabla_{\boldsymbol{p}}\tilde{g}_m^s\|_1 = \sum_{i\in\{L,S\}}\sum_{j=1}^M |\nabla_{p_j^i}\tilde{g}_m^s|$, which can be derived as

$$\frac{1}{\sum\limits_{i\in\{L,S\}}\sum\limits_{j=1}^M e^{u_j^i(p_j^i - p_m^s)}} + \frac{2p_m^s \sum\limits_{\substack{i\in\{L,S\}, j=1,\ldots,M \\ i\neq s \text{ or } j\neq m}} u_j^i e^{u_j^i(p_j^i - p_m^s)}}{\Big(\sum\limits_{i\in\{L,S\}}\sum\limits_{j=1}^M e^{u_j^i(p_j^i - p_m^s)}\Big)^2}. \quad (20)$$

For the case $p_m^s > \max\{\mathcal{P}_m^s\}$, we can find from (19) that the approximation error approaches zero if $\sum_{i\in\{L,S\}}\sum_{j=1}^M e^{u_j^i(p_j^i - p_m^s)}$ approaches one. Since $e^{u_j^i(p_j^i - p_m^s)} = 1$ when $i = s$ and $j = m$, we know that $\sum_{i\in\{L,S\}}\sum_{j=1}^M e^{u_j^i(p_j^i - p_m^s)}$ approaching one means that $e^{u_j^i(p_j^i - p_m^s)}$ should approach zero for any $i \neq s$ or $j \neq m$. When this happens, it can be found from (20) that the gradient magnitude $\|\nabla_{\boldsymbol{p}}\tilde{g}_m^s\|_1$ approaches one, thus avoiding the gradient vanishing or exploding. To achieve such a desired result with zero approximation error, we can choose u_m^s as 0 for p_m^s and choose u_j^i as $\frac{-\rho}{p_j^i - p_m^s}$ for $p_j^i \neq p_m^s$, where ρ is a large positive constant.

For the case $p_m^s < \max\{\mathcal{P}_m^s\}$, we observe from (19) that the approximation error approaches zero as $\sum_{i\in\{L,S\}}\sum_{j=1}^M e^{u_j^i(p_j^i - p_m^s)} \to \infty$. However, this will lead to a vanishing gradient as shown in (20), necessitating the optimization of the smoothing parameter $\boldsymbol{u}$. The problem of optimizing $\boldsymbol{u}$ can be formulated similar to (17) as

$$\min_{\boldsymbol{u}\succeq 0} \ |\tilde{g}_m^s - g_m^s| \quad (21a)$$

$$\text{s.t. } \|\nabla_{\boldsymbol{p}}\tilde{g}_m^s\|_1 = \bar{V}, \quad (21b)$$

where $\bar{V}$ is the gradient requirement varying with the number of iterations, and $\succeq$ is the element-wise inequality.

Problem (21) has a complicated objective function, making it difficult to obtain its global optimal solution. Yet, as analyzed in Appendix C, the second term of $\|\nabla_{\boldsymbol{p}}\tilde{g}_m^s\|_1$ in (20) is very small when the objective function (21a) is minimized. By omitting this term, we obtain an approximate solution for the smoothing parameters as derived in Appendix C, which is

$$u_j^i = \begin{cases} \frac{1}{p_j^i - p_m^s}\log\frac{1/\bar{V} - 2M + |\mathcal{G}|}{|\mathcal{G}|}, & \frac{1}{1+|\mathcal{G}|} \geq \bar{V}, p_j^i > p_m^s, \\ \frac{-\rho}{p_j^i - p_m^s}, & \frac{1}{1+|\mathcal{G}|} < \bar{V}, p_j^i < p_m^s, \\ 0, & \text{else}, \end{cases} \quad (22)$$

where $|\mathcal{G}|$ is the size of the set $\mathcal{G} \triangleq \{p_j^i | p_j^i > p_m^s, i \in \{L,S\}, j = 1,\ldots,M\}$.

C. Nonlinear Penalty to Constraint Violation

A constrained optimization problem can be solved through learning by transforming it into a primal-dual form [19, 20]. The introduced Lagrangian multipliers are learned by DNNs, referred to as multiplier DNNs.

The training of multiplier DNNs is related with the values of c_m^s in (15), since the gradients of the objective function in (15a) with respect to multipliers λ_m^s is c_m^s. The sign of c_m^s determines the update direction of λ_m^s, while the magnitude of c_m^s affects the update step size. If a constraint is nearly met, the magnitude of c_m^s will be very small, resulting in a slow update of the multiplier DNN. As a result, the multiplier is hard to increase rapidly enough to penalize the violated constraints, thereby causing a high probability of constraint violation.

The SBT has a very stringent reliability requirement, which is reflected by the very low decoding error probability ϵ_m in the considered problem. When learning based methods are employed, the learned policy may not always satisfy constraints, and hence the probability of constraint violation, denoted as $\hat{\epsilon}_m$, also affects the achieved reliability. The total error probability can be expressed as

$$1 - (1 - \hat{\epsilon}_m)(1 - \epsilon_m) \approx \hat{\epsilon}_m + \epsilon_m, \quad (23)$$

where the approximation holds since both $\hat{\epsilon}_m$ and ϵ_m are very small. It indicates that the learning based method should ensure a very low level of constraint violations.

To reduce constraint violation, the penalty for minor violations of constraints should be increased. We address this by introducing a nonlinear function $q(\cdot)$ on c_m^s in (15) to amplify the penalty of constraint violations. With $q(\cdot)$ and the smoothing functions, the Lagrangian function in (15a) can be expressed as

$$\sum_{f=1}^F \tilde{\mathbf{1}}(\tilde{g}_f, v) + \sum_{s\in\{L,S\}}\sum_{m=1}^M \lambda_m^s q(c_m^s). \quad (24)$$

The nonlinear function $q\left(\cdot\right)$ is designed as

$$q\left(c_m^s\right)=\begin{cases}\kappa\big(\frac{2}{1+e^{-wc_m^s}}-1\big), & c_m^s>0,\\ -\min\big\{\frac{\lambda_m^s}{2},1\big\}, & c_m^s\leq 0.\end{cases} \tag{25}$$

The principle of the design and its impact on the penalty of constraint violation are explained as follows.

When $c_m^s>0$, i.e., a constraint violation happens, $q\left(c_m^s\right)$ is designed as $\kappa\big(\frac{2}{1+e^{-wc_m^s}}-1\big)$, which is a smoothed indicator function $\tilde{\mathbf{1}}(c_m^s)$ as defined in (16b), where κ is a constant. Such a $q\left(c_m^s\right)$ causes a sharp increase of the violation value from c_m^s to $\kappa\big(\frac{2}{1+e^{-wc_m^s}}-1\big)\rightarrow\kappa$. The smoothing parameter w is computed with the previously proposed adaptive smoothing parameter method, which can avoid the gradient vanishing and gradient exploding caused by introducing the nonlinear function $q\left(\cdot\right)$. On the other hand, when the constraint is satisfied, $q\left(c_m^s\right)$ is designed as $-\min\{\frac{\lambda_m^s}{2},1\}$. For small λ_m^s, say less than two, $q\left(\cdot\right)$ equals to $-\frac{\lambda_m^s}{2}$. Upon substituting it into (24), we can find that the gradient with respect to the multiplier is $-\lambda_m^s$, i.e., we reduce the multipliers based on their current values. To prevent an over reduction of multipliers, we restrict that the decreasing rate of multipliers does not exceed one, which is realized by the minimal function $\min\{\frac{\lambda_m^s}{2},1\}$.

Remark: When applying the nonlinear function $q\left(c_m^s\right)$ to c_m^s in the Lagrangian function (24), we have transformed the constraint from $c_m^s\leq 0$ to $q\left(c_m^s\right)\leq 0$. Such a transformation is equivalent, because it can be found from (25) that $q\left(c_m^s\right)\leq 0$ always implies $c_m^s\leq 0$.

D. Proposed Learning Method

With the previously developed two smoothing functions and the nonlinear penalty function, we are ready to present our learning method.

According to the proof in [20], the primal-dual formulation (15) can be equivalently transformed as the following functional optimization problem

$$\max_{\lambda_m^s(\boldsymbol{\gamma})}\min_{p_{m,f}^s(\boldsymbol{\gamma})}\mathbb{E}\Big\{\sum_{f=1}^{F}\mathbf{1}\left(g_f,v\right)+\sum_{s\in\{L,S\}}\sum_{m=1}^{M}\lambda_m^s(\boldsymbol{\gamma})\,c_m^s\Big\} \tag{26a}$$

$$\text{s.t.}\quad\sum_{s\in\{L,S\}}\sum_{f=1}^{F}p_{m,f}^s(\boldsymbol{\gamma})\leq P_{\max},m=1,\ldots,M, \tag{26b}$$

$$\boldsymbol{p}\left(\boldsymbol{\gamma}\right),\boldsymbol{\lambda}^s\left(\boldsymbol{\gamma}\right)\succeq 0,s\in\{L,S\}, \tag{26c}$$

where $p_{m,f}^s(\boldsymbol{\gamma})$ and $\lambda_m^s(\boldsymbol{\gamma})$ are the functions of the channel gains $\boldsymbol{\gamma}\triangleq[\gamma_{1,1},\cdots,\gamma_{M,F}]\in\mathbb{R}^{1\times MF}$, defining $\lambda_m^s(\boldsymbol{\gamma})$ as a function is necessary because the constraints in (14b) and (14c) should be satisfied for all the possible values of $\boldsymbol{\gamma}$, $\mathbb{E}\{\cdot\}$ denotes the expectation taken over $\boldsymbol{\gamma}$, $\boldsymbol{p}\left(\boldsymbol{\gamma}\right)=\left[p_{1,1}^L(\boldsymbol{\gamma}),\cdots,p_{M,F}^S(\boldsymbol{\gamma})\right]\in\mathbb{R}^{1\times 2MF}$, and $\boldsymbol{\lambda}^s(\boldsymbol{\gamma})=[\lambda_1^s(\boldsymbol{\gamma}),\cdots,\lambda_M^s(\boldsymbol{\gamma})]\in\mathbb{R}^{1\times M}$.

We employ DNNs to parameterize the functions $\boldsymbol{p}\left(\boldsymbol{\gamma}\right)$ and $\boldsymbol{\lambda}^s(\boldsymbol{\gamma})$. Instead of learning directly from channel gains $\boldsymbol{\gamma}$, we consider a sorted version of $\boldsymbol{\gamma}$. As pointed out by [26], if the inputs and outputs of a DNN are permutation equivariant along a dimension, then sorting the inputs along that dimension can significantly reduce the required training samples.

In our problem, the input channel gains and output power allocations are permutation equivariant along the dimension of RB. Specifically, reordering the channel gains associated with the RBs leads to a corresponding reordering in the optimal power allocation. For the single-user scenario, sorting the channel gains is straightforward since each RB has a single channel gain. For the multiuser scenario, however, sorting becomes difficult because each RB has multiple channel gains from different users. We present a heuristic sorting method for the multiuser scenario, which is shown to be effective by simulation results later. The method is inspired by the multiuser RB selection strategy given in Sec. III-B, which operates as follows. First, the RB on which the first user has the largest channel gain among F RBs is selected as the first RB. Next, the RB on which the second user has the largest channel gain among the remaining $F-1$ RBs is selected as the second RB. This sequential process continues until all RBs are sorted.

After the sorting of the channel gains $\boldsymbol{\gamma}$, the power allocation policy, i.e., the mapping from the sorted $\boldsymbol{\gamma}$ to the corresponding power allocation $\boldsymbol{p}$, is parameterized by a *policy DNN* $\boldsymbol{p}\left(\boldsymbol{\gamma};\boldsymbol{\theta}_p\right)$, where $\boldsymbol{\gamma}$ is the input and $\boldsymbol{\theta}_p$ is the trainable parameter. The functions $\boldsymbol{\lambda}^s(\boldsymbol{\gamma})$, $s\in\{L,S\}$ are parameterized by two *multiplier DNNs*, $\boldsymbol{\lambda}^s\left(\boldsymbol{\gamma};\boldsymbol{\theta}_s\right)$, $s\in\{L,S\}$ respectively, where $\boldsymbol{\theta}_s$ is the trainable parameter. With the parameterization, problem (26) is converted into the optimization with respect to the trainable parameters $\boldsymbol{\theta}_s$ and $\boldsymbol{\theta}_p$. Further considering the previously proposed two smoothing functions and the nonlinear penalty function, they are obtained from the following optimization problem

$$\max_{\boldsymbol{\theta}_s}\min_{\boldsymbol{\theta}_p}\mathbb{E}\Big\{\sum_{f=1}^{F}\tilde{\mathbf{1}}\left(\tilde{g}_f,v\right)+\sum_{s\in\{L,S\}}\sum_{m=1}^{M}\lambda_m^s(\boldsymbol{\gamma};\boldsymbol{\theta}_s)\,q\big(c_m^s\big)\Big\} \tag{27a}$$

$$\text{s.t.}\quad\sum_{s\in\{L,S\}}\sum_{f=1}^{F}p_{m,f}^s(\boldsymbol{\gamma};\boldsymbol{\theta}_p)\leq P_{\max},m=1,\ldots,M, \tag{27b}$$

$$\boldsymbol{p}\left(\boldsymbol{\gamma};\boldsymbol{\theta}_p\right),\boldsymbol{\lambda}^s\left(\boldsymbol{\gamma};\boldsymbol{\theta}_s\right)\succeq 0,s\in\{L,S\}. \tag{27c}$$

The training of the policy DNN and two multiplier DNNs is implemented by the primal-dual stochastic gradient method. The primal parameters $\boldsymbol{\theta}_p$ and the dual parameters $\boldsymbol{\theta}_L$ and $\boldsymbol{\theta}_S$ are iteratively updated along the descent and ascent directions of the sample-averaged gradients of the loss function $\mathcal{L}$, respectively, where $\mathcal{L}=\sum_{f=1}^{F}\tilde{\mathbf{1}}\left(\tilde{g}_f,v\right)+\sum_{s\in\{L,S\}}\sum_{m=1}^{M}\lambda_m^s(\boldsymbol{\gamma};\boldsymbol{\theta}_s)\,q\big(c_m^s\big)$. Constraints in (27b) and (27c) can be satisfied by properly choosing the activation function at the output layer of both the policy DNN and multiplier DNNs.

In Fig. 3, we illustrate the training process for the policy and multiplier DNNs as well as the inference process with the well-trained policy DNN, i.e., offline training and online inference. During each iteration, we first sort the samples in a batch based on the sorting method presented in this subsection. Next, we compute and update the smoothing parameters in two smoothing functions and a nonlinear penalty function based on the methods proposed in the previous subsections.